

1. ISO/IEC 12207:2010: Жизненный цикл ПО. Группы процессов ЖЦ.

Официальная классификация процессов программной инженерии задается международным стандартом ISO/IEC 12207:2008 "Systems and Software Engineering — Software Life Cycle Processes" и его российским аналогом ГОСТ Р ИСО/МЭК 12207-2010

В этих стандартах процессы привязываются к понятию "Жизненный цикл программного обеспечения". В авторитетном словаре программной инженерии ISO/IEC/IEEE 24765-2010 "Systems and Software Engineering — Vocabulary" дано следующее определение: Жизненным циклом программного обеспечения называется период времени, который начинается с момента замысла программного продукта и заканчивается в момент, когда ПО больше недоступно для использования.

Процесс жизненного цикла в общем случае представляет из себя определенную работу/совокупность действий над входными данными и ресурсами, на выходе после действий будут выходные данные и ресурсы. Сама совокупность действий не определена конкретно.

Действия, которые могут выполняться во время жизненного цикла ПО распределены по двум категориям. Одна из них называется "Процессы в контексте системы" описывает процессы для работы с непосредственно программным продуктом. Другая категория называется "Специальные процессы программных средств" и она содержит в себе процессы в отношении программного продукта, являющегося частью более крупной системы.

Процессы в первой категории делятся на группы "Процессы соглашения", "Процессы организационного обеспечения проекта", "Процессы проекта" и "Технические процессы"

- Процессы соглашения нужны для выработки соглашений между стороной заказчика и стороной разработки
- Процессы организационного обеспечения проекта нужны для организации ресурсов и инфраструктуры для обеспечения удовлетворения организационных целей и установленных соглашений
- Процессы проекта нужны для планирования, выполнения и оценки и управления продвижением проекта, а также выполнение специализированных целей менеджмента
- Технические процессы. Тут все просто, они обеспечивают определение требований к системе, их преобразование в полезный программный продукт, его применение и изъятие из обращения, если он не используется

Процессы во второй категории делятся на группы "Процессы реализации программных средств", "Процессы поддержки программных средств", "Процессы повторного использования программных средств". Как уже было озвучено ранее, эта категория описывает процессы в отношении программного продукта, являющегося частью более крупной системы, чем рассматриваемая.

Любой крупной компании-разработчику следует следовать этим процессам для обеспечения надлежащего качества разработки. Однако хочется отметить, что процесс разработки не является застывшим предписанием, которому команда должна следовать догматически. Напротив, он должен быть очень гибким и адаптивным (к проблеме, проекту, команде и культуре организации). Таким образом, процесс, адаптированный для одного проекта, может (и будет) существенно отличаться от

процесса, адаптированного для другого проекта. Если модели процессов применяются догматически и без какой-либо адаптации, проект рискует лишь утонуть в бюрократии без какого-либо преимущества для всех заинтересованных лиц.

2. Модели ЖЦ (последовательная, инкрементная, эволюционная).

Модель жизненного цикла программного обеспечения — это структура, определяющая последовательность выполнения и взаимосвязи процессов, действий и задач на протяжении всего жизненного цикла ПО.

Существует несколько моделей ЖЦ (они же стратегии разработки ПО):

- Последовательная (она же “Однократный проход”, Waterfall): в этой модели заранее определены все требования, это позволяет провести все стадии разработки последовательно и один раз. Получившиеся результаты окончательны и, как правило, не подлежат пересмотру (например, проектирование марсохода, после того, как мы его отправим в космос, мы уже не сможем поменять его внутреннюю начинку), но даже если и допустима фаза сопровождения программного продукта, обратная связь требует возврата к самой начальной стадии (и поэтому ее ожидание может занимать какое-то время). Тем не менее, для такой модели легко спрогнозировать и предъявить заказчику стоимость и сроки разработки.
- Инкрементная: здесь нам также заранее известны все требования, однако мы разбиваем продукт на несколько эквивалентных (с архитектурной точки зрения) частей и производим разработку в несколько этапов. Первая версия реализует часть запланированных возможностей, следующая вводит дополнительные возможности и т. д. пока не получим готовый продукт.
- Эволюционная: в реальном мире часто бывает такое, что на старте нам известны далеко не все требования к продукту, и эволюционная модель ЖЦ решает эту проблему: система также строится в виде последовательности версий, но требования уточняются по ходу разработки.

В реальной разработке используется инкрементно-эволюционный подход, поскольку чисто инкрементной или чисто эволюционной стратегии развития ПО практически не существует.

Сравнительная таблица моделей ЖЦ:

Стратегия разработки	В начале процесса определены все требования?	Множество циклов разработки?	Промежуточное ПО распространяется?
Однократный проход	Да	Нет	Нет (отсутствует)
Инкрементная (запланированное улучшение продукта)	Да	Да	Может быть
Эволюционная	Нет	Да	Да

3. Водопадная (каскадная) модель

Разработана приблизительно в 60-х годах и описана Уинстоном Ройсом в 70-х. Суть заключается в том, что у нас есть 7 фаз разработки ПО и есть итерации между ними, в том порядке, в котором они

перечислены:

- Стадия сбора требований к системе
- Стадия сбора требований к ПО
- Стадия анализа требований
- Стадия проектирования дизайна ПО
- Стадия кодирования
- Стадия тестирования
- Стадия эксплуатации системы

Имеется возможность откатиться к предыдущей фазе, но подчеркивается сложность этого действия. Так, например, тестирование — это первая фаза, на которой может быть обнаружено, что ожидаемые характеристики (функциональность, производительность, объем данных и др.) отличаются от утвержденных в результате анализа, в результате чего придется полностью пересмотреть либо требования, либо дизайн системы, что приводит к глобальной переделке системы и потенциально к двукратному (!!) росту сроков и стоимости разработки.

4. Методология Ройса.

без схемы, нет смысла ее учить

Методология Ройса берет за основу каскадную модель, однако существенно дорабатывает ее главный недостаток: высокую рискованность (потенциальное глобальное переделывание системы), вводя 5 дополнительных шагов:

Сначала дизайн программы. Ройс определил стадию предварительного дизайна ПО между сбором требований и их анализом. Несмотря на то, что из-за отсутствия анализа дизайнер не может учесть все требования, в его силах убедиться, а могут ли они быть вообще реализованы. К этой стадии не допускается участие программистов и аналитиков. В ней предлагается спроектировать, определить и создать модель обработки данных, а также создать документ-обзор системы.

Документирование. Ройс включает важные документы процесса разработки:

- Требования к системе
- Спецификация предварительного дизайна
- Спецификация дизайна интерфейсов
- Финальная спецификация дизайна системы
- План тестирования
- Инструкция пользовательской эксплуатации

Делай это дважды. Ройс предлагает параллельно с основной разработкой создавать упрощенную версию системы с ускоренными сроками разработки и использовать ее в качестве пилотной версии.

Это позволяет подтвердить или опровергнуть основные характеристики ПО.

Контроль тестирования. Поскольку тестирование это наиболее рискованная фаза с точки зрения денег и сроков, ему должно быть уделено особое внимание. При планировании тестирования из процесса тестирования должен быть исключен дизайнер системы, должна быть проведена "визуальная инспекция" (повторный просмотр кода другим лицом, которое без глубокого анализа выявит проблемы в логических путях и/или заметные визуальные дефекты). После исправления простых ошибок провести проверку ПО в тестовом окружении.

Подключение пользователя. Во время разработки Ройс предложил давать возможность наблюдения непосредственно пользователю конечной системы. Для этого выделено 3 точки: предварительный, критически и финальный просмотр. На каждой из этих точек пользователю дается возможность получить пользовательский опыт, его оценку и подтверждение дальнейшего продвижения разработки.

Схему рисовать гроб =(Но она буквально повторяет текстовое описание, приведенное выше.

5. Традиционная V-chart model J.Munson, B.Boehm.

За основу V-chart модели была взята каскадная модель, однако ее слегка изогнули в V-образную форму, сопоставив почти каждой фазе из каскадной модели соответствующее тестирование (отсюда и название — V-chart)

- Концепт системы ↔ Использование и поддержка

Линия валидации концепта

- Сбор требований и планирование ↔ Установка системы в конечном окружении

Граница сбора требований + валидации и верификации. Ниже — верификация.

- Дизайн продукта ↔ Приемочное тестирование
- Детализация дизайна ↔ Системное и интеграционное тестирование
- Кодирование ↔ Unit-тестирование

Динамическое тестирование включает компьютерное исполнение тестов, а статическое тестирование предполагает проверку артефактов разработки без их компьютерного исполнения, например, спецификации, технических решений, дизайна и пр.

6. Многопроходная модель (Incremental model).

Частный случай V-chart модели. На этапе разработки происходит вместо одного этапа несколько, каждый включающий в себя:

- Детальное проектирование
- Кодирование
- Интеграция прототипа

Разработка становится более управляемой и видимой для заказчика, но есть и недостатки. Архитектура системы и многие ее части стремятся к устареванию и деградированию, то есть частям системы будет

требоваться полный и/или частичный рефакторинг за счет средств заказчика. А из-за высокой скорости изменений поддерживать документацию в больших проектах становится также достаточно сложно. Помимо этого, присуще сложности с заключением контракта и оцениванием стоимости разработки программного продукта.

7. Модель прототипирования (80-е).

Суть данной модели заключается в том, что мы сначала быстро спланируем всю итерацию, быстро проанализируем требования и подходы к реализации, разработаем прототип, покажем его заказчику. И если заказчику все понравится, то мы разрабатываем окончательную версию ПО, тестируем и внедряем ПО, далее сопровождаем его. В ином случае делаем новый прототип, новый интерфейс и базу данных, пока заказчик не скажет, что его все устраивает.

В современных методах разработки применяются развитые средства макетирования для создания такого рода прототипов.

8. RAD методология (??) (достался мне на экзамене)

Rapid Application Development порождена корпорацией IBM в 1980-е годы, обнародована в 1991. Идея заключается в том, что в те года была очень слабо развита автоматизация бизнес-процессов, программы писались в текстовом редакторе, база данных была доступна практически в ручном режиме, и все это в целом было очень грустно.

Решением проблемы заключалось создание интегрированных сред разработки, а методология заключается в том, что после небольшого планирования из готовых компонентов, как из кубиков, собирается ПО.

Пользователь принимает непосредственное участие в процессе разработки. Да, это не всегда отличается продуманным интерфейсом или удобством разработки, но это позволяет очень быстро разработать функциональность, которая была нужна "уже вчера".

Сегодня таковыми примерами можно считать Computer Aided Software Engineering (CASE)-системы, 1C предприятие (паралимпийские игры по программированию), Oracle E-Business Suite. В них можно создавать формы данных, выполнять запросы к системе с помощью пользовательского интерфейса.

9. Спиральная модель

В начале 80-х, проанализировав все существующие модели, Б. Боем сформулировал метод разработки крупного программного обеспечения, который получил название спиральной модели.

Ключевой особенностью метода стало включение в него постоянной оценки изменений и рисков. На каждой итерации спирали при переходе между четвертями мы анализируем риски — и проект может не перейти к следующей стадии, если он не прошел риск-анализ.

10. UML Диаграммы: Структурные и поведенческие.

UML — графический язык моделирования общего назначения, предназначенный для спецификации, визуализации, проектирования и документирования артефактов, создаваемых при разработке программного продукта.

UML диаграммы делятся на структурные и поведенческие:

- К структурным диаграммам относятся доменные модели (они же диаграммы классов), диаграмма развертывания. Эти диаграммы нужны для статического описания системы, демонстрации ее архитектуры, конфигурации или специфических элементов предметной области.
- К поведенческим диаграммам относятся диаграммы прецедентов использования (они же Use-Case диаграммы), диаграммы последовательностей/состояний. Они представляют собой динамическую картину действий, происходящих в системе.

11. UML: Use-case модель.

Use-case модель отражает требования к программному продукту и представляет из себя диаграмму вариантов использования программного продукта.

Главным элементом модели является действующее лицо (actor) — пользователь, который непосредственно взаимодействует с системой. Под пользователем указывается его роль. Эктор никогда не является частью системы. Под эктором тоже можно считать время (в ряде случаев)

Прецедент содержит в себе действие, выполняемое в системе (например, "Пользователь вводит логин и пароль". Прецеденты одних модулей могут выступать экторами для других модулей.

Ассоциация используется в диаграмме прецедентов для указания того, что эктор использует систему и выполняет указанный прецедент.

Также в Use-Case модели используются следующие стереотипы:

- Стереотип Include (включение) организует иерархию прецедентов использования системы и позволяет включать одну общую деятельность в несколько Use-Case'ов.
- Стереотип Extend (точка расширения функциональности) указывает, собственно, на точку расширения функциональности. Например, при просрочке возврата книги в библиотеке с пользователя взимут штраф. То есть при наличии определенных условий будет применена дополнительная функциональность. Не стоит путать со стереотипом Generalization.
- Стереотип Generalization (обобщение) указывает на отношение is a (A является B), показывающая, что данный объект является потомком более высокого класса объектов.

Напоследок, в Use-Case диаграмме используется элемент System Boundary (граница системы). Он представляет интерес, когда есть несколько подсистем, и требуется знать разделение между ними.

12. UML: Диаграмма классов.

Диаграмма классов (она же доменная модель) является примером структурной диаграммы и отображает в первом приближении реализованную предметную область. Например, мы рисовали такую диаграмму на 1 курсе к каждой лабораторной работе по программированию на Java. Преимущество такой диаграммы заключается в ее наглядности.

13. UML: Диаграмма последовательностей (???)

Диаграмма состояний (диаграмма конечных автоматов) является примером поведенческой диаграммы и отображает модель "Finite state machine" (программа конечных автоматов), представляющая собой набор состояний и условием переходов между ними.

14. UML: Диаграмма размещения

Диаграмма размещения является примером структурной диаграммы и отображает физическую архитектуру размещения программного обеспечения, а также возможности его использования. Например, на этой диаграмме может быть нарисована серверная машина, в которой имеется сервер приложений Java (прим. ред.: фу), в нем крутится наша лаба по веб-программированию, а также может иметься пользовательская машина с запущенным в ней браузером, и соединение происходит по протоколу связи HTTPS.

15. *UP методологии (90-е). RUP: основы процесса.

- Unified Process методологии берут за собой идею о создании четкой унифицированной структуры создания программного продукта, подходящей для любой команды людей, позволяющей создать с ее помощью любой программный продукт.

Одна из таких методологий носит название Rational Unified Process. Одноименную компанию зарегистрировали в 90-е годы. Вся разработка представляла из себя инкрементно-эволюционный процесс, в котором все было разбито на фазы. Помимо них, в RUP также имеют место быть т. н. дисциплины. Они представляют собой набор подходов, правил и указаний, которые необходимы для решения той или иной задачи, например, анализ требований. Все дисциплины созданы с целью конкретизировать действия разработчиков для того, чтобы максимально избежать сюрпризов и конкретизировать процесс разработки.

Всего есть 4 фазы: Inception (Начало), Elaboration (проектирование), Construction (построение), Transition (внедрение). В рамках инкрементного подхода фазы могут делиться на итерации.

В RUP четко определено, как должен строиться весь процесс. Явно описаны все роли, которые принимают участие в создании программного продукта (около 30 ролей).

Роль — группа обязанностей, которую берет на себя определенный участник команды. У каждой роли есть набор деятельности. На входе и выходе деятельности используются, создаются и модифицируются артефакты. Артефактом в разработке принято называть любой результат труда роли, например, диаграмма, документ, программный код и пр. Деятельность четко регламентирована.

В отличие от других моделей ЖЦ, в RUP каждый процесс детально описан и связан с другими процессами. Сам по себе RUP представляет собой программный продукт, оформленный в виде веб-приложения на апплетах Java (прим. ред.: фу) с гиперссылками.

16. RUP: Фаза «Начало».

Цели этой фазы:

- Оценить проект: граница, сценарии использования.
- Понять, какие ресурсы и время нужно на него выделить.

- Оценить риски, подготовить окружение.

Веха "Lifecycle objects":

- Формируется артефакт Vision (концепция).
- Стороны договорились о сроках, первоначальной стоимости, требованиях, технологиях, приоритетах (???).

17. RUP: Фаза «Проектирование».

Цели этой фазы:

- Финализировать архитектуру системы.
- Разработать прототипы на основании архитектуры
- Убедиться, что сроки и стоимость приемлемы и стабильны, риски оценены и учтены.
- Убедиться, что архитектура стыкуется со сроками и стоимостью, и на ее основании программный продукт реализуем.

Веха "Lifecycle architecture":

- Концепция, требования и архитектура проекта соблюдены?
- Сформированы критерии тестирования прототипов?
- Тестирование прототипа показало отсутствие основных рисков?
- Сроки и стоимость все еще актуальны?
- Стороны поддерживают продолжение проекта?

18. RUP: Фаза «Построение».

Цели этой фазы:

- Экономически эффективно, в максимально короткие сроки реализовать программное обеспечение.
- Итеративно провести анализ, проектирование, разработку и тестирование продукта, создать необходимые выпуски (альфа, бета...).
- Подготовить продукт, окружение и пользователей к развертыванию.

Веха "Initial Operation Capability":

- Достаточно ли стабилен выпуск для передачи заказчику?
- Готова ли принимающая сторона?
- Актуальны ли еще расценки на программный продукт?

19. RUP: Фаза «Внедрение».

Цели этой фазы:

- Развернуть систему, базу данных на целевой платформе
- Обучить пользователей пользоваться продуктом
- Запустить маркетинг и продажи, каналы сбыта
- Отладить сбои, выкатить хотфиксы и пр.
- Убедиться в самодостаточности пользователей
- Провести открытый анализ со всеми стейкхолдерами на соответствие программного продукта заявленной концепции

Веха "Release":

- Пользователи удовлетворены?
- Финальная стоимость разработки приемлема?

20. Манифест Agile (2001).

"Мы постоянно открываем для себя более совершенные методы разработки ПО, непосредственно этим занимаясь и помогая в этом другим. За это время мы выяснили что:

- Люди и взаимодействие важнее процессов и инструментов.
- Работающий продукт важнее исчерпывающей документации.
- Сотрудничество с заказчиком важнее заключенного согласованного контракта.
- Готовность к изменениям важнее следования первоначальному плану.

То есть, не отрицая важности всего того, что справа, мы отдаем предпочтение тому, что слева."

Думаю, это исчерпывающе описывает данный манифест. Однако хочется добавить, что он во многом повторяет концепцию, названную "Духом RUP". Дух RUP зародился, как ни странно в Rational Unified Process и описывает отношении к организации процесса и взаимодействия с заказчиком.

Данный манифест в дальнейшем породил другие методологии разработки, как на основе RUP (AUP), так и не на его основе (Scrum).

21. Scrum.

Scrum относится к представителям гибких методологий разработки ПО, разработанным в 1986 году, популярным в нулевые годы. Основой этой методологии является так называемый артефакт Backlog (бэклог — долг), в который заказчик записывает функциональность, которую он хочет видеть в продукте, отсортированной в порядке приоритетов, задаваемых самим заказчиком, с оценкой трудоемкости их реализации.

Существуют бэклог продукта и бэклог спринта, отличающиеся уровнем детализации. Также имеет место быть артефакт “инкремент продукта”.

Разработка делится на спринты. Спринтом называют период времени от 1-2 до 4 недель. За это время разработчики реализуют выбранный набор требований из бэклога. Каждый спринт заканчивается демо-версией, которую оценивает заказчик. В течение нескольких спринтов производится ретроспектива с целью оптимизации рабочего процесса (перераспределение обязанностей и пр.).

Команда в Scrum не очень большая (3-10 человек), также среди них выделяют особую роль — владелец продукта (Product Owner), он определяет порядок реализаций требований из бэклога (часто, он и является их автором). Для каждого спринта задачи конкретизируются и передаются на разработку команде. Также выделяется роль Scrum Master’a, он ответственен за проведение Scrum-митинга, который помогает команде планировать спринт и следит за внутренними отношениями в команде.

Scrum-митинг проходит ежедневно утром, где каждый участник команды отчитывается о проделанной работе и имеющихся сложностях, а также том, что он планирует сделать к следующей встрече.

22. Disciplined Agile 2.X (2013). (??)

С введением гибких методологий разработки, очень быстро выяснилось, что они плохо масштабируются на команды с большим числом участников, которые привыкли вставать на работу в 7 утра и не отличаются высокой мотивацией, какой, например, обладают владельцы и участники стартапа.

Disciplined Agile 2.X является относительно свежим решением на рынке и берет по своей сути лучшее из двух миров *UP и Agile процессов. Деление на фазы и дисциплины было частично взято из RUP (“Начало”, “Построение” и “Внедрение”), но основной цикл разработки построен на базе гибких методологий (в частности, Scrum).

Помимо озвученного, Disciplined Agile 2.X предлагает процессы, выходящие за рамки процесса разработки, например, управление архитектурой и повторным использованием кода, управление персоналом, непрерывное улучшение процессов разработки и т. д.

23. Требования. Иерархия требований.

Требование — условия или возможности, которым должна удовлетворять система. А также это подробное описание того, что должно быть реализовано. Важно, что требование не описывает, как именно оно должно быть реализовано.

Пример требования в ПО: “SEC0 Система должна обеспечивать доступ к любой загруженной в нее информации по запросу федеральных органов власти США”

В RUP требования описывается с помощью шаблона Software Requirement Specification, она же SRS, а также модели прецедентов для более наглядного представления.

Обычно у заказчика нет четкого набора требований. У него есть потребности. По своей сути это представляет собой пожелания, которые решают конкретную задачу. К сожалению, заказчик обладает огромным магическим мышлением, поэтому его представления о ПО заканчиваются на наборе кнопочек, которые каким-то образом что-то делают. Так, например, заказчик может сформулировать потребность следующим образом: “Мне нужна сковородка, и чтобы на ней не пригорала еда!”

Далее в дело вступает аналитик и в рамках предметной области преобразовывает потребности в набор функций, которые должна реализовывать система. Они все еще не являются полноценными требованиями, но при этом являются куда более содержательными, без потери смысла для заказчика (т. е. они все еще написаны понятным для него языком). Характерный пример функции: "Сковородка должна иметь антипригарное покрытие и термоизолирующую ручку".

После утверждения набора функций с заказчиком можно переходить непосредственно к формулированию требований, включающих подробное конкретное описание будущего программного продукта. Например: "Диаметр сковородки должен быть 25см, величина бортиков 5см, она должна быть выполнена из сплава алюминия с кремнием марки АК7П (7% кремния).

24. Свойства и типы требований (FURPS+).

Требования должны быть:

- Корректными
- Однозначными
- Полными
- Непротиворечивыми
- Поддающиеся приоритизации
- Проверяемыми
- Модифицируемыми
- Отслеживаемыми

Иными словами, требования должны быть хорошими, а плохими не должны быть =). Ну, в самом деле, они должны быть корректными и полными, не противоречить друг другу. Любые требования должны поддаваться трекингу, с возможностью в любой момент модифицировать его, в том числе модифицировать приоритет.

В RUP также используется классификация требований FURPS+:

- Functional — определяют, что именно должна делать система (+ часто требования по безопасности)
- Non-Functional — определяют характеристики и ограничения системы
- Usability — требования по UX
- Reliability — требования по надежности
- Performance — требования по производительности
- Supportability — требования по расширяемости и масштабируемости

(+) Дополнительные

- Ограничение архитектуры

- Требования к реализации и интерфейсу
- Физические требования

25. Формулирование требований. Функциональные требования.

Требование имеет четкую структуру:

```
<id> <система> должна/shall <требование>
```

Требование всегда нумеруется, должно иметь имя системы и, непосредственно, само требование, удовлетворяющее свойствам требований.

Функциональные требования нужны, чтобы описать, что именно должна делать система. Они описывают возможности и функциональность программного продукта, а также требования по безопасности.

Пример: "FR0 Система должна иметь возможность просмотра образовательных статей" / "SEC0 Система должна обеспечивать доступ к любой загруженной в нее информации по запросу федеральных органов власти США"

26. Требования к удобству использования и надежности.

Требования по юзабилити нужны для:

- Учета особенностей пользователя, например, лица с ограниченными возможностями
- Эстетических требований
- Согласованности UI
- Описания справочной подсистемы и пользовательской документации
- Учебных материалов и мастеров ПО для упрощения UX

Пример: U0 "Система должна иметь внешнее оформление в соответствии с brandbook'ом корпорации заказчика"

Требования по надежности нужны для:

- Обеспечения взаимодействия со случаями отказа системы
- Способности восстановления продуктивной части системы после отказа
- Точности и предсказуемости ее поведения

Пример: "R0 Система должна обеспечивать максимальное время простоя на уровне 15 минут в год с учетом планового технического обслуживания"

27. Требования к производительности и поддерживаемости.

Требования по производительности нужны для:

- Определения скорости и эффективности решения задач
- Определения готовности системы решить поставленную задачу
- Регулирования пропускной способности, времени отклика, восстановления и т. п.
- А также ограничение на потребляемые ресурсы

Пример: "P0 Система должна рассчитывать время чтения статьи не дольше, чем за 2 секунды"

Требования по расширяемости нужны для:

- Обеспечение поддерживаемости и совместимости системы
- Возможности адаптировать систему под разные конкретные задачи
- Способности конфигурировать систему, например, локализация
- А также указывает ограничения на реальную продакшн платформу

Пример: "S2 Система должна быть совместима с UNIX-серверной вычислительной мощностью"

28. Атрибуты требований.

Существуют разные атрибуты требований:

Приоритет (правило MoSCoW):

- MUST - Minimal Usable SubseT — фундаментальное требование, без него продукт не имеет смысла
- Should have — важное требование, было бы здорово, если бы его реализовали
- Could have — не важное требование, первый кандидат на удаление, потенциально важное требование
- Won't have — эти требования могут быть реализованы в следующих версиях
- И/или приоритет (цифрами)

Статус:

- Предложено
- Согласовано
- Включено
- Отклонено
- Трудоемкость (человекочасы, попугаи, иные метрики)

Стабильность — насколько требование не подвержено изменениям

- Низкая
- Средняя
- Высокая
- Целевая версия внедрения
- Риски

29. Описание прецедента.

При разработке диаграмму прецедентов используют лишь для первого приближения модели. Все прецеденты трансформируются в требования в виде их текстового описания, GUI или иных способов, а получившийся артефакт называют "Описание прецедента". Например:

30. Риски. Типы Рисков.

Риск — это фактор и событие, потенциально негативно влияющее на проект. Анализ рисков пришел к нам в сферу программной инженерии из бизнес-сферы. Ввел их Б. Боэм, он же автор спиральной модели разработки ПО.

Типы рисков:

Прямые — ими можно управлять

- Ресурсные: люди, время, финансы, организация. Пример: "Васю бросила девушка и на работу он больше не выходит в течение недели".
- Бизнес-Риски: конкуренция, подрядчики, убыточность решения. Пример: "Корпорация "Рога и копыта" внезапно выпускает продукт-убийцу, делая бесполезной ваш программный продукт, и ваша команда может смело отправляться искать новую работу".
- Технические риски: границы проекта, технологические, внешних проектных зависимостей. Пример: "У нас прод упал", "В библиотеке, которую мы используем, обнаружили уязвимость нулевого дня, караул", "Петя устроился работать к нам Senior разработчиком, а вместо этого он бегаёт и ищет себе замену за 70% его зарплаты"
- Политические риски: сферы влияния менеджмента. Пример: "Экстренные новости, арестован заместитель министра обороны Тимур Иванов, ему вменяют взятку в особо крупном размере..."

Непрямые — с ними уже ничего не поделаешь =(Такова судьба

- Форс-мажор. Пример: "На здание, где по удивительному стечению обстоятельств находилась в этот момент вся команда разработки и поддержки, упал метеорит. Никто не выжил..."

31. Управления рисками. Деятельности, связанные с оценкой.

Оценка риска включает в себя:

- Идентификацию риска

- Анализ риска
- Приоретизацию риска

Как идентифицировать риск? Марвин Дж. Карр предложил строить таксономию источников рисков. По сути своей это дерево, где на первом уровне расположены классы рисков, то, где они непосредственно могут возникнуть, далее в классе определены конкретные риски в виде элементов и у каждого риска есть атрибуты, так и ищем потенциальные источники рисков.

Риск идентифицирован. Как его анализировать? Существуют разные математические модели и их визуализация (например, графическая), однако два основных параметра риска это его вероятность и масштаб (магнитуда) потерь. Они могут задаваться в разных величинах и очень субъективно.

Риски проанализированы. Что теперь? Вычисляем т. н. экспозицию риска: произведение вероятности риска на масштаб потерь. Полученные риски сортируем по экспозиции. Тадам, уроки приоретизации для самых маленьких.

32. Управления рисками. Деятельности, связанные контролем и управлением.

Контроль и управление риском включает в себя:

- Планирование реакции
- Мониторинг рисков
- Разрешение неопределенностей

Что можно сделать с риском, который потенциально может наступить? Самое первое и логичное, что может придти на ум — попытаться избежать риск. Для этого необходимо разработать комплекс мер по отсрочке, исключению или хотя бы уменьшению вероятности наступления риска. Иногда мы можем перевалить этот риск как горячую картошку на кого-нибудь еще (и мы с удовольствием это сделаем). Например, запросим дополнительное финансирование у заказчика и тогда он рискует получить проект не в срок =) Ну и третий вариант — это принять риск и потери с ним связанные. Такова судьба, а менеджмент лучше распускать.

Во время разработки риски необходимо постоянно переоценивать и вести т. н. мониторинг рисков. Список может быть довольно большим, поэтому контролируем первые топ-10 самых деструктивных и вероятных рисков. Часто применяют в помощь автоматизацию, например, Jira.

Для того, чтобы риск не наступил, нужно строить большое число прототипов системы, использование моделирования, сбор постоянной обратной связи от заказчика и пользователей, так мы будем разрешать неопределенности еще на этапе их зарождения, и минимизируем их вероятности. Также в помощь уменьшения рисков будет непрерывная работа над ошибками, постоянная рефлексия специального отдела контроля качества =), и грамотная работа отдела по набору персонала.

33. Изменение. Общая модель управления изменениями.

Изменение это контролируемое, журналируемое обновление системы.

У сложных систем изменения нарастают как снежный ком и их контролирование представляет собой очень серьезную проблему в менеджменте.

Все начинается с того, что наш любимый пользователь хочет внести какое-то улучшение в систему или сообщить о какой-то проблеме в ней. Формируется артефакт требований или отчета о проблеме. Все это формирует т. н. запрос на изменение, который порождает запись в логе/журнале изменений системы.

Далее запросы на изменение попадают к менеджеру проекта. Умный менеджер смотрит на запрос и дает ему техническую оценку: насколько изменение осуществимо и сколько это будет стоить, далее специальный комитет на этом основании оценивает: оправдано ли вообще это изменение (например, проблемы одного пользователя часто в расчет не берутся, если же пользователей станет много, тогда изменение будет реализовано с куда большей вероятностью). Все это отражается в журнале изменений.

Если изменение оказалось действительно важным и нужным, оно попадает в отдел по работе с изменениями, которые проведут оценку того, как это изменение влияет на ее другие части. К сожалению, в большой системе невозможно, чтобы изменения в одной части никоим образом не повлияли на другую часть, так что данный вопрос нуждается в серьезном анализе.

После этого мы можем оценить ресурсы, требуемые для осуществления изменения, спланировать процесс изменений и создать график работ. Затем мы реализуем изменения, создаем соответствующие артефакты, в совокупности реализующие требуемые изменения. После этого мы тестируем систему, обновляем документацию, и выпускаем новую итерацию. При этом создаются артефакты отчета о тестировании, новая документация и выпуск новой версии системы.

В завершение изменения передаются проектному менеджеру, который оценивает и верифицирует изменения, и утверждает окончательную версию изменения, закрывая его в журнале. Конец.

34. Системы контроля версий. Одновременная модификация файлов.

Сами по себе системы контроля версий требуются для обеспечения одновременного доступа к файлам и программному коду нескольких человек, а также контроль над самими изменениями, которые вносят программисты в исходный код.

Существуют 3 вида систем контроля версий:

- На основе файловой системы. Название говорит само за себя, такие СКВ создавали файлы слежения за директорией и такие системы работали только в рамках одной файловой системы. Устаревший подход.
- Централизованные. В данных СКВ есть единый центральный узел, с которым разработчики обмениваются изменениями по специальному протоколу, сервер ведет полный их учет. Главный и очевидный недостаток заключается в том, что если центральный узел упал, это автоматически вызывает караул у всего отдела руководства и горькие слезы у программистов, которым не посчастливилось с этим столкнуться. Пример: Subversion (прим. ред.: svn govno)
- Распределенные. В них разработчики делятся между собой полной версией репозитория, каждый может претендовать на роль центрального узла. Обратно изменения попадают после нескольких стадий локальных проверок. Пример: Git.

Главной проблемой СКВ является разрешение ситуаций, когда несколько человек работают над одним файлом. И есть 2 подхода по разрешению таких ситуаций:

Lock-modify-unlock

Тут все просто. Кто первый занял файл и захватил его блокировку — тот молодец, остальные могут, как вариант, сделать копию файла, и работать в ней. Недостатки, это снижает командный дух, если программист сидит за файлом слишком долго. Возможно, потенциальным решением здесь будет работать над файлами ночью =) Но это будет работать до момента, пока до столь умного решения не догадается еще один разработчик, и проблема снова встанет столбом.

Copy-modify-merge

Тут же мы себе копируем весь репозиторий, вольны вносить какие угодно изменения, далее вся команда собирается вместе и в конце рабочего дня задает один главный вопрос: "50 конфликтных мест в файле, как же нам так удалось это сделать и как с этим жить?". Собственно, при всех преимуществах, решаемых над первым подходом, этот подход даровал нам конфликты при слиянии рабочих копий.

35. Subversion. Архитектура системы и репозиторий.

Централизованная система контроля версий Subversion (прим. ред.: svn govno) имеет несколько уровней. Начинается все с централизованного репозитория, который физически может иметь реализации в файловой системе или Berkley DB.

Далее к репозиторию обеспечивается определенный уровень доступа. Он может быть доступен с помощью демона svnserve или с помощью сервера Apache, их возможности эквивалентны.

Удаленный доступ к репозиторию осуществляется по протоколу svn, еще более удаленный доступ (не в рамках одной локальной машины) осуществляется связкой https+svn/ssh+svn.

На том конце провода нас ждет клиент SVN, который не только обменивается файлами с центральным узлом, но и управляет локальной копией файлов.

Репозиторий в Subversion (прим. ред.: svn govno) это набор файлов проекта, организованный определенным образом для удобной работы с ним. Каждая фиксация увеличивает версию репозитория на 1. Версия репозитория это всегда целое число. Гарантируется, что при успешной фиксации репозиторий целостный.

- trunk — основная ветвь разработки
- branches — хранение модификаций и вех продукта (релизы, выполнение фич без влияния на trunk)
- tag — функционально целостные изменения (хотфиксы и пр.)

Отдельно хочется упомянуть, что в Subversion (прим. ред.: svn govno) нет понятия "ветки", таковыми здесь выступают подкаталоги, а все файлы копируются целиком при создании новой "ветки".

36. Subversion: Основной цикл разработчика. Команды.

- svn checkout — получение первоначальной рабочей копии

- Утром: пресс качат обновить рабочую копию (svn update)
- Днем: вносить изменения (svn add delete mkdir move copy)
- Вечером: просмотреть сделанное (svn diff и svn status). Откатить изменения (svn revert)
- Ночь!: загрузить изменения других коллег (svn update), разрешить конфликты, сделать фиксацию (svn commit)

37. Subversion: Конфликты. Слияние изменений.

В Subversion (прим. ред.: svn govno) при обновлении копии (svn update) могут возникать конфликты.

Самый частый конфликт — конфликт содержимого файла. Это когда один и тот же файл менялся разными разработчиками (например, по-другому поменяли одну и ту же строку).

Как правило, в этом случае изменение откладывают на потом (postpone), так как редко в этом случае вы можете что-то делать самостоятельно. Следует найти разработчика, с которым возникло конфликтное изменение и согласовать с ним разрешение конфликта. Однако вы можете принять изменения извне, переопределить их своими изменениями, открыть редактор и поменять все руками, запустить внешнее средство и пр.

При выборе postpone создаются три файла. filename.mine - до update — локальный файл), а также filename.rOLDREV и filename.rNEWREV — файл из удаленного узла до и после правок. С помощью triple diff утилит сравнивать эти файлы и разрешать конфликты. Часто есть современные IDE, которые позволяют в интерактивном режиме разрешать конфликты.

Еще из типа конфликтов выделяют конфликты структуры. Например, в основном репозитории перемешали файлы, а в локальной копии были сделаны в них изменения (очень мерзко, согласен). Конфликт решается обновив руками структуру согласно тому, что прилетело из удаленного узла, так и пообщавшись с тем, кто сотворил сея изменения в структуре репозитория (желательно, уволив его после этого).

При работе над параллельными ветками, часто требуется применить все изменения из одной ветки в другую. Для этого существует команда svn merge. Она подгружает изменения из "веток" (каталогов, в svn нет веток) в рабочую копию с учетом изменения структуры. При слиянии возможны конфликты, которые ничем не отличаются от указанных выше.

38. GIT: Архитектура и команды. (достался мне на экзамене)

Git — децентрализованная* система контроля версий, придуманная самим Линусом Торвальдсом, поскольку в момент разработки линукса он поссорился с создателем СКВ, на которой в тот момент базировалось ядро линукса.

Архитектура гита подразумевает существование всей копии репозитория у каждого разработчика. Каждый разработчик независимо вносит какие-то правки в репозиторий и фиксирует в него же изменения. Если же надо отправить изменения другому разработчику, Git предоставляет удобный интерфейс для их передачи (see git patch, GitHub)

Таким образом, каждый разработчик сам выбирает, какие изменения брать, а какие нет, чтобы работать с репозиторием и исходным кодом. Git — это очень гибкая и удобная система контроля версий,

недаром она сейчас является доминирующей на рынке.

В git book выделяют три основных сценария рабочего процесса с Git'ом:

- Централизованный рабочий процесс: есть централизованный репозиторий (например, на GitHub'e), все разработчики синхронизируют работу именно с ним.
- Рабочий процесс с менеджером по интеграции: каждый разработчик имеет канонический локальный репозиторий. Каждый открывает к нему доступ для остальных только для чтения, правки вносятся в локальном репозитории, потом запрашивается у менеджера по интеграции слияние правок в его каноническую репозиторию.
- Рабочий процесс с лейтинантами и диктатором. Самый известный пример: разработка ядра Linux. Лейтинанты — это интеграционные менеджеры: каждый отвечает за свою конкретную часть. У лейтинантов есть свой интеграционный менеджер — диктатор, его репозиторий считается эталонным, он принимает окончательное решение о внесении правок. Считай говоря, мы сделали промежуточный слой в виде лейтинантов, чтобы сотни тысяч остальных ребят могли вносить вклад в репозиторий.

Чтобы начать работать с git, вам пригодится команда `git init`. Или можно клонировать чужой репозиторий по его URL командой `git clone`.

В git имеется три зоны, где может находиться файл: `unstaged (modified) area`, `stage area`, `committed area`. Переход от `unstaged area` в `staged area` достигается командой `git add`. Обратный переход тоже возможен, для этого в помощь команды `git restore --staged` или `git reset`.

Переход из `staged area` в `committed area` достигается командой `git commit`. После фиксации мы можем просто поменять файл для его перехода в `unstaged area`, чтобы вернуть его в `staged area`, мы можем отменить коммит, переместив указатель HEAD на один коммит назад (`git reset --hard`)

После того, как файл вышел из `modified area`, изменения навсегда останутся в git, и они никуда не исчезнут без стороннего вмешательства в `.git` каталог. Это является большим преимуществом Git'a, его архитектура серьезно уберегает нас от нечаянного удаления файла и других нелепых ситуаций. Посмотреть состояние репозиторий можно командой `git status`.

У каждого коммита есть хеш, имя автора коммита и его e-mail. Это важно, поскольку ветки в git это не больше, чем указатели на коммиты, а сами по себе коммиты складываются в граф древовидной* структуры. В отличие от `svn` (прим. ред.: `svn govno`), создание новых веток в гите это не больше, чем сделать файл с указателем на 40 символов хеша. Поэтому ветки здесь создаются почти на каждый чих (и это считается `best-practices`).

При работе с ветками, git предоставляет удобную команду `git branch` (`git checkout`, последний вообще швейцарский нож в гите, но ответ на вопрос и так уже огромный), позволяющую создавать новые ветки или переключаться между существующими. Ветки можно сливать одну в другую, командой `git merge`.

Также для централизованной работы, например на сервисе GitHub вам будут полезны команды `git fetch`, `git pull` (она по сути делает `git fetch origin/<branch> + git merge <branch>`). На гитхабе доступна функциональность запросов на вытягивание (применение слияний между ветками разных репозиторий, NB!!! Это не функциональность самого гита), а также клонирование репозитория себе на аккаунт (это называется `fork`, тоже не функциональность самого гита!!!). Таким образом, разработчик

может форкнуть репозиторий (fork), клонировать его себе на машину (git clone), применить изменения (git add, mv,...), сделать фиксацию (git commit), отправить в локальный репозиторий (git push), и сделать pull request автору.

Напоследок, всю древовидную структуру репозитория можно получить командой git log (--graph), сравнить два коммита можно командой git diff.

39. GIT: Организация ветвей репозитория.

В Git создавать ветку принято на почти любое самостоятельное изменение кода (и это считается best-practice). Ветки в гите легкие — это просто указатель из 40 символов, являющиеся ничем иным как хеш коммита в древовидной структуре репозитория.

При создании репозитория, автоматически создается ветка master (main с 2020 года), которая является как правило эталонной в нашем репозитории. Далее как правило происходит ответвление в ветку dev, где происходит основной процесс разработки, в master как правило лежит только релизный образец исходного кода (на помощь здесь приходит CI/CD). Каждая фича принята, чтобы разрабатывалась в отдельной ветке.

Когда накапливается достаточно изменений, все ветки собираются воедино, происходит слияние, решение конфликтов, тестирование, и из новых фич собирается релиз, который и является новой вехой в разработке программного продукта. Ветвь hotfix иногда бывает нужна, чтобы оперативно поправить критические баги, которые по каким-то причинам не были покрыты тестированием (человек не идеален). Изменения оттуда напрямую коммитятся в мастер и создают новый релиз, пусть и меньшей итерации (v.x.x.n+1)

Процесс повторяется для всех новых функциональных требований.

40. GIT: Плагин git-flow.

Каждое проделываемое действие в гите может быть достаточно громоздким, поэтому над гитом была создана абстракция, которая получила название git flow. Это по своей сути плагин, который предоставляет интерфейс гита в виде управления версиями без точного знания CKB Git.

Вся структура команд git flow уместается на данной схеме:

Все это позволяет пользоваться репозиторием Git на более высоком уровне, независимым от синтаксиса git образом. Никто, правда, этим не пользуется, если быть честным....

41. Системы автоматической сборки: предпосылки появления

Ужас №1: рутинный процесс сборки.

При сборке необходимо повторять одинаковую последовательность команд:

```
javac -d build -cp lib src/*.java
jar -cfm ...
```

Как вариант, можно сделать скрипт. Однако он будет

- Платформенно-зависимым
- Очень большим и иметь перспективы стать еще больше

Ужас №2: зависимость от конкретной платформы

- Собранная программа под одну машину не будет работать на другой
- Собранная программа под одну операционную систему не будет работать на другой

Ужас №3: медленная сборка

- Сборка может занимать сутки, файлов с исходным кодом может быть тысячи, они могут быть большие.
- Не обязательно компилировать все файлы: давайте только те, которые были изменены с учетом их зависимостей.
- Нет параллелизма — и современное оборудование простаивает

42. Системы сборки: Make и Makefile.

Make считается средством сборки низкого уровня (по мнению автора, это декларативное средство сборки, но не стоит его путать с Maven, в этой классификации оно императивное). Представляет из себя стандартную утилиту, включенную в Unix-подобные системы. Суть в следующем. Программисту надо написать файл сборки (makefile), в котором надо описать опции компиляции и зависимости, которые будут резолвиться при сборке, а также пути к стандартным компиляторам, средствам запаковки и прочие команды для терминала, которые будут исполняться в рамках одного таргета.

Синтаксис makefile'а выглядит примерно следующим образом:

Когда мы запускаем утилиту make, она смотрит наличие makefile в текущем рабочем каталоге (pwd), смотрит в этом мейкфайле наличие таргета по-умолчанию (all), далее она строит и пробегает по графу зависимостей и с учетом времени модификации производит последовательность действий, необходимых для сборки конечного продукта.

Таким образом, Make — это язык, который в неявном виде позволяет определить последовательность действий при сборке, а также императивно указать последовательность действий, необходимую и достаточную для того, чтобы произвести сборку продукта. К сожалению, ее простота не совмещается с удобством, программисту нужно держать в голове достаточное количество макропеременных и действий по-умолчанию, определенных в системе.

43. Системы сборки: Ant. Команды Ant.

Apache Ant является императивным средством сборки программ, написанных на языке Java (прим. ред.: фу). Весь процесс управляется при помощи файла build.xml. Его структура выглядит примерно следующим образом:

XML это не лучшее решение для императивного описания сборки. Однако он очень хорош для машинной верификации, поэтому разработчики Ant решили использовать именно его. Идейно, Ant очень похож на Make, оба представляют системы сборки одного класса.

Имеется возможность вызывать цели явно, не через атрибут `depends`. (`antcall`). Эти цели могут быть объявлены в другом файле, поэтому присутствует возможность разбивать сборку на модули.

Имеется поддержка неизменяемых значений — свойств. Задаваться могут как в `build.xml`, так и в переменных окружения среды, так и в `.property` файлах.

Некоторые команды из Ant:

- `checksum` — создает контрольную сумму для файлов
- `chmod` — изменить права доступа к файлу
- `copy`, `delete`, `mkdir`, `move` — команды файловой системы
- `javac`, `java`, `jar`, `javadoc` — команды JDK
- `exec`, `sleep`, `waitfor` — команды исполнения среды
- `echo` — вывести в лог сообщение
- `scp`, `ftp` — команды передачи по сети
- `junit`, `junitplatform` — команды платформы для запуска тестов (JUnit 4 & 5)
- `bzip2`, `tar`, `war`, `zip` — команды архиваторов

Многие из этих команд зачем-то повторяют стандартные команды в операционной системе. Однако не все так просто, эти команды поднимают нас на уровень выше, делая процесс сборки независимым от конкретной платформы. Везде, где запускается Apache Ant, запустится процесс сборки с исходным `build.xml`. За это дружно благодарим “истинную” кроссплатформенность джавы.

44. Системы сборки: Ant-ivy.

В связи с развитием интернета (и огромным числом легаси проектов, которые по объективным причинам имели Ant скрипты сборки на 10+ МБ), Ant смог заполучить себе менеджер зависимостей. Сторонние библиотеки встраиваются в сборку автоматически, и загружаются из центрального репозитория (Maven), а менеджерит всем этим хозяйством утилита Ivy.

Таким образом, это существенно упрощает работу с third-party библиотеками, а процесс их встраивания в `build.xml` очень простой и понятный:

45. Системы сборки: Maven. POM. Репозитории и зависимости.

Maven является логическим продолжением автоматических средств сборки проекта. Императивные средства хороши до того момента, пока мы в состоянии проследить полный цикл сборки программы, но для больших проектов это не всегда выполнимая задача. Часто мы хотим сказать: что именно

собирать, а не как это надо делать. И мир нуждался в системах, которые знали, как надо собирать исходный код.

Так появились декларативные средства сборки, и Maven является одним из его представителей. Maven представляет декларативную систему сборки программ, написанных на языке Java (прим. ред.: фу). С другой стороны, у нас появилось развитие интернета и система репозиторий, где хранятся все сторонние библиотеки на все случаи жизни, и мы указав, что хотим видеть их в нашем проекте, просто получаем их из репозитория и пользуемся ими.

Декларативная магия описывается, так же, как и в Ant'e, в XML-файле. Только в отличие от Ant'овского XML, в этом мы описываем объектную модель проекта (Project Object Model — POM). Она содержит в себе

- Имя, версию и тип проекта
- Местоположение исходного кода
- Зависимости (сторонние, внутренние)
- Плагины — средства, используемые во время разработки (они дополнительно знают, как работать с исходным кодом, чтобы его декларативно собрать)
- Профили — альтернативные конфигурации проекта

Репозитории и зависимости принципиально имеют следующую схему:

Указывая в pom.xml зависимости (1), мы даем знать утилите Maven информацию о том, за какими внешними библиотеками надо обратиться к центральному узлу (2). Получив стороннюю библиотеку определенной версии (3), она загружается в локальное хранилище (3). Далее происходит сборка (4) с учетом предоставленного кода, и готовый программный продукт помещается в каталог target (5).

46. Maven: Структура проекта. GAV.

Структура проекта Maven в виду декларативных ограничений фиксированная:

- target — рабочая и целевая директория
- src/main — основные исходные файлы
- src/main/java — каталог исходных файлов Java (прим. ред.: фу)
- src/main/webapp — раздаточное содержимое web-страниц (прим. ред.: буэ)
- src/main/resources — встраиваемые ресурсы, не нуждающиеся в компиляции
- src/test — исходные файлы тестов
- src/test/java — каталог исходных файлов тестов Java (прим. ред.: фу)
- src/test/resources — встраиваемые тестовые ресурсы, не нуждающиеся в компиляции

Система наименования модулей в Maven строится по принципу GAV — GroupID:ArtifactID:Version. Пример:

47. Maven: Зависимости. Жизненный цикл сборки. Плагины.

В Maven транзитивные зависимости! Это означает, что если сторонняя библиотека зависит от еще одной библиотеки, то исходный проект тоже будет зависеть от этой библиотеки.

Зависимости состоят из:

- GAV-описания
- Scope
- `compile` — зависимая библиотека будет участвовать в компиляции и дальнейшей загрузке
- `provided` — зависимая библиотека будет участвовать при линковке, но не будет включена в итоговую сборку. Используется в энтерпрайзе, где сервер приложений часто уже обладает нужными зависимостями самостоятельно.
- `test` — используется только в тестовом окружении
- Иные: `runtime`, `system`

Type:

- `jar`
- `pom`
- `war`
- `ear`
- `zip`

Зависимости со скоупом `compile` и типом `jar` являются зависимостями по умолчанию.

Жизненный цикл Maven-приложения:

- `generate-sources/generate-resources` — эта фаза сгенерирует недостающие исходные файлы, если есть соответствующее правило DSL
- `compileJava` — скомпилировать основные исходные файлы
- `testCompileJava` — скомпилировать тестовые файлы
- `test` — протестировать исходный код
- `package` — собрать скомпилированные файлы в пакет (в `Jar` или `War` архив)
- `integration-test (if any, pre and post)` — запустить интеграционный тест
- `install` — установка в локальном репозитории
- `deploy` — если есть сервер приложений, загрузить собранный артефакт туда

Плагины в Maven управляют процессом сборки программы. Они знают, как именно собирать исходный код. И они встраиваются в определенный этап ЖЦ, например, packaging плагины могут выступать способы упаковки кода в jar или war архив.

48. Системы сборки: Maven. POM. Репозитории и зависимости. Системы сборки: Gradle. Преимущества и файл сборки.

Gradle — современная модная молодежная система сборки.

- Нейтральна к языкам программирования
- Можно несложно написать сборку любого приложения на любом языке программирования
- Подобно Maven базируется на плагинах + жизненном цикле
- Основные задачи по декларативной сборке описаны в плагинах
- Плагины определены для многих типов проектов
- No more XML! Скрипт сборки пишется на адекватном, человеческом DSL, а именно, на Kotlin (прим. ред.: <3 <3<3<3<3), можно и на Groovy
- Инкрементальная и параллельная сборка

Пример build.gradle.kts:

49. Системы сборки: GNU autotools. Создание конфигурации проекта.

Речь идет о том, что происходит вне мира Java (прим. ред.: фу). Здесь имеется огромное число платформенно-зависимого кода, с которым нам приходится иметь дело. И как способ выручить нас на помощь приходит утилита GNU autotools.

Мы имеем огромную базу кода, многих из которых является платформеннозависимым. С помощью утилиты autoscan мы проводим диагностику и сканирование исходных кодов на предмет таких платформеннозависимых вызовов, и на выходе получаем файл configure.scan. Далее нам нужно доработать эти файлы вручную (configure.ac).

Также мы создаем makefile.am, где в явном виде указываем названия исполняемых программ, из каких исходников они должны быть собраны, а также зависимости. Такой файл создается в каждом подкаталоге базы исходного кода.

Далее мы пробегаемся утилитами autoheader, который создаст шаблон заголовочного файла, а команда aslocal проверит, что установлено на компьютере разработчика, и посмотрит, что из этого используется в проекте.

Затем утилитами automake и autoconf создаются конечные makefile.in и configure файлы, которые уже будут использоваться в процессе конфигурации на конечной машине.

Принципиальная схема:

50. Системы сборки: GNU autotools. Конфигурация и сборка проекта.

На этапе создания конфигурации мы получили исполняемый файл `configure`. При его запуске на конечной системе будет произведена проверка наличия нужных исполняемых файлов (компиляторов и пр.). На выходе мы будем иметь платформеннозависимый `makefile` и конфигурационный файл системы или информацию о том, что каких-то компонентов не хватает, и их нужно доустановить.

Далее происходит обычная работа с `make`: `make all + sudo make install` установят платформеннозависимый софт на конечную систему.

Принципиальная схема:

51. Сервера сборки/непрерывной интеграции.

Для выполнения автоматической сборки существуют специальные автоматизированные сервера, политики непрерывной интеграции. Они очень гибкие в своей настройке и позволяют делать практически что угодно. Однако вот пример полезного использования CI/CD:

Новый коммит в master ветке? Собираем проект, генерируем артефакты, документацию, бинарные файлы для целевой системы, запускаем тесты, создаем отчеты, рассылаем их по электронной почте. Политика запускается каждый день в 00:00, после окончания рабочего дня.

Теперь мы можем с утра открыть отчеты, увидеть, как у нас все успешно собралось (ага, конечно) и мы можем с этим как-то работать.

Примеры: GitHub Actions, CruiseControl, Jenkins, Travis CI, GitLab CI/CD.

52. Основные понятия тестирования. Цели тестирования.

Основные понятия и цели тестирования задаются международной классификацией квалификации тестировщиков (International Software Testing Qualification Board - ISTQB)

Поясним за понятия:

- Mistake — ошибка человека
- Fault — изъян в работе ПО, неправильный шаг алгоритма
- Failure — внешнее проявление дефекта, отказ, сбой
- Error — невозможность выполнить задачу вследствие отказа
- BUG — неформальное использование. Может обозначать: дефект, изъян, сбой, невозможность выполнить задачу, что-то другое или вовсе ничего не обозначать.

Поясним за цели:

Основная цель: повысить уровень пользовательского доверия к вашему программному обеспечению, дать пользователю понять, что Ваша программа работает корректно во всех необходимых обстоятельствах.

А также:

- Обнаружение и предотвращение дефектов
- Предоставление информации о работе вашего кода

N/B! "Тестирование почти никогда не способно доказать отсутствие ошибок" (с)

53. Понятие полного тестового покрытия и его достижимости. Пример.

Полное тестовое покрытие гласит о том, что код покрыт всеми возможными тестовыми кейсами.

Например, для проверки корректности работы умножителя двух 32 битовых чисел нам потребуется протестировать всевозможные результаты умножений для всех вариантов чисел A (2^{32} чисел) и B (2^{32} чисел) → 2^{64} вариантов. Пусть мы получили каким-то (альтернативным способом) тестовую базу на 2^{64} записей для каждого варианта умножения. Пусть эта запись содержит два числа A, B (32 бита каждый), ожидаемый результат (64 бита), а также результат проверки теста (8 бит). Итого это 132 бита на каждую запись. Всего таких записей 2^{64} → ~ 3.13 петабайта будет занимать таблица с полной тестовой базой для нашего умножителя.

Пусть мы нашли хранилище для этого бессмысленного и беспощадного занятия. Пусть у нас одна проверка будет занимать 1 такт машинного времени (магия). Тогда нам надо будет прогнать 2^{64} тактов на вычислительной технике. Пусть нам дан процессор на 3ГГц. Тогда проверка нашего умножителя будет занимать приблизительно 180 лет. И это речь идет о достаточно примитивной схемотехнике и алгоритме.

Считается, что полное тестовое покрытие недостижимо.

54. Статическое и динамическое тестирование.

Статическое тестирование иногда называют иногда рецензированием. Его суть заключается в том, что нам вовсе необязательно иметь рабочую архитектуру и рабочий код + разработанный набор тестов для того, чтобы выполнить тестирование. Статическому тестированию можно подвергнуть не только программный код, но и другие артефакты, например, спецификацию. Проводится в ручном и автоматизированном режиме, например:

Ручное статическое тестирование:

- Звонок (опытному) другу — "Помоги найти мне ошибку..."
- Инспекция — Несколько человек, которым дали простую работу, будут делать ее хорошо

Автоматическое статическое тестирование:

- Линтеры
- Автокоррекция в ворде =>

Динамическое тестирование подразумевает созданный набор тестов, набор модулей, групп модулей, потенциально, всей системы. Однако недавно был предложен подход к разработке при помощи предварительного создания тестов (Test Driven Development). В общем говоря, динамическое тестирование подразумевает запуск кода, в этом их принципиальная разница от рецензирования.

55. Автоматизация тестов и ручное тестирование.

Автоматизация тестов это хороший способ избавиться от монотонной рутинной работы ручных тестировщиков, но часто бывает такая ситуация, что у компании на это попросту не хватает ресурсов, и поэтому мы можем нанять максимально неквалифицированный персонал, который сможет нам штамповать ручные тесты пачками ежедневно. Тем не менее, чем больше компания и ее продукт по размерам, тем больше нужно включать процесс автоматизации в тестирование.

Вообще говоря, автоматизированное тестирование вовсе не сокращает ручной труд, ведь эти тесты надо кому-то написать. К автоматизации приравнивают регрессионное тестирование, которое призвано проверять, что изменения в системе не повлияли на то, что работало до этого, а также тестирование совместимости, которое проверяет продукт в разных окружениях. Это достаточно сложный процесс, который ставит острым углом вопрос о количестве тестов, ведь чем их меньше, тем легче будет общая сложность и нагрузка на команду тестировщиков.

56. Источники данных для тестирования. Роли и деятельности в тестировании.

Выделяют два (три) классических подхода к тестированию:

- Метод “черного ящика”. Исходные данные для тестов берутся из спецификации, требований, информации об архитектуре. Взаимодействие идет через публичный интерфейс системы и мы совершенно ничего не знаем о том, как это работает под капотом
- Метод “белого ящика”. Исходные данные опираются на основании анализа исходного кода. Исходный код анализируется и представляется в виде алгоритмического графа, из которого вычленяются места, которые жизненно необходимо покрыть тестами.
- (также) распространен метод “серого ящика”. При нем мы имеем лишь частичное представление о структуре работы кода, преимущественно применяя методики тестирования “черного ящика”.

Дополнительно источниками данных для тестов могут быть модели (UML) и жизненный опыт тестировщика, который может заранее иметь на уме потенциально опасные места.

Роли и деятельности в тестировании делятся на несколько больших категорий:

Проектирование тестов

Высококвалифицированные специалисты, разбирающиеся в предметной области, дискретной математике и программировании

Автоматизация тестов

Непосредственно разработчики тестов. Принято, что модульные тесты пишут сами разработчики исходного кода, а интеграционные и системные тесты пишут отдельно выделенные для этого программисты

Исполнение тестов

Исполнители тестов это очень аккуратные и внимательные люди, выполняющие работу четко по инструкции, которую им разработали проектировщики. Не требуют особой квалификации и их задача

просто аккуратно провести тестирование системы.

Анализ результатов

Аналитики тестовых результатов должны быть ознакомлены с предметной областью, в которой они работают, а также ряд технических знаний и их задача вразумить в себя полученные на прошлом этапе результаты тестирования.

Этот список был предоставлен для того, чтобы развеять миф о том, что на “тестируемых” не требуются какие-либо знания и умения, а также тестирование является куда менее престижной деятельностью, чем разработка.

57. Понятие тестового случая и сценария.

Тестовый случай представляет из себя набор входных тестовых значений, преобразователя тестовых данных (наша программа) и набора выходных значений (ожидаемый результат).

К значениям относятся данные + условия. В ожидаемом результате описано то, что мы ожидаем увидеть в качестве выходных данных, постуловий + иные последствия и/или изменения, которые происходят в программе.

Ожидаемый результат обязан быть определен до начала теста, иначе правдоподобные, но некорректные результаты будут засчитаны. Особенно здесь стоит подход TDD, при котором тесты определяются заранее и разработка считается завершенной тогда и только тогда, когда успешно выполнены все тесты.

Тестовый случай должен быть повторяем, как следствие, автоматизируемым. То есть при повторном запуске тестового случая мы не должны рассчитывать на изменение результатов. Это особенно важно, когда речь идет о регрессионном тестировании.

Тестовый случай должен учитывать состояния. Это расширяет тестовое покрытие и дает возможность классифицировать тесты по состояниям и собирать из этого тестовые сценарии. В Российской официальной терминологии под “сценарием” имеется в виду тестовый “случай”, и это создает путаницу в кругу разработчиков.

Тестовый сценарий это последовательность тестовых случаев.

Пример: Банкомат, типичное использование системы.

#	Начальное состояние	Ввод	Действие системы	Вывод	Итоговое состояние
1	Готов	Пользователь вставляет карточку	Успешное чтение карты	Приглашение ко вводу PIN-кода	Ожидание PIN
2	Ожидание PIN	Пользователь вводит верный PIN	Успешная проверка PIN-кода	Приглашение к действию с картой	Ожидание действия
3	Ожидание действия	Пользователь выбирает опцию	Проверка баланса,	Выдача 5000 рублей	Выдача денег и

#	Начальное состояние	Ввод	Действие системы	Вывод	Итоговое состояние
		"Снять с карты 5000 рублей"	пересчет денег		карты
4	Выдача денег и карты	Пользователь забирает деньги и карту	Завершение выдачи	Благодарность за использование	Готов

Тестовый сценарий должен помимо этого отрабатывать некорректные или ошибочные варианты:

#	Начальное состояние	Ввод	Действие системы	Вывод	Итоговое состояние
3	Ожидание действия	Пользователь выбирает опцию "Снять с карты 5000 рублей"	Проверка баланса, пересчет денег	Выдача 5000 рублей	Выдача денег и карты
4	Выдача денег и карты	Пользователь оставил деньги и карту в банкомате в течение 45 секунд.	Возврат денег на счет и в банкомат, блокировка карты	Сообщение об ошибке, информирование о необходимости посещения банка	Готов

58. Выбор тестового покрытия и количества тестов. Анализ эквивалентности. (??)

При выборе количества тестов надо ориентироваться на следующий баланс:

- Больше тестов → больше тестовое покрытие → выше качество и доверие
- Меньше тестов → продукт разрабатывается быстрее → продукт быстрее выйдет на рынок

Зачем нужно делать больше тестов +- очевидно, но вот отказываться от большего покрытия не лишено смысла. Очень часто вы можете столкнуться с суровыми реалиями конкурентной среды, где конкуренты дышат вам в спину и только спят и думают, как же отобрать у вас дорогого покупателя. Поэтому очень часто скорость выхода на рынок имеет очень серьезное влияние на разработку. Кстати, поэтому первые версии продуктов и получаются сырые (Superpunk 2077), хоть это и приводит к тому, что каких-то пользователей вы рискуете потерять, как правило, эти риски не настолько существенны по сравнению с тем, что на рынок выйдет ваш конкурент и вам ничего по итогу не останется.

С количеством тестов определились, что с самими тестами? Существуют несколько методов выбора тестового покрытия:

Партиции эквивалентности (эквивалентное разбиение)

В рамках этого метода производится анализ исследуемой функции на наличие участков, где она ведет себя примерно одинаковым способом.

Все, что нам осталось, это проставить тесты на граничных значениях, и думать, что мы все сделали правильно.

Таблица альтернативных решений

Строим таблицу комбинаций входных и выходных данных, на этом основании строим тестовые сценарии, покрывающие группы этих сочетаний

Таблица переходов

Все тоже самое, что и в таблице альтернативных решений, только вместо комбинаций входных и выходных данных используем состояния (конечные автоматы), а покрываем тестами переходы между состояниями

Сценарии использования

В качестве топлива для тестового покрытия могут быть разработанные ранее сценарии использования. При этом в каждый описанный сценарий вводятся конкретные (!!) значения, которые мог бы подать пользователь. Обычно каждому такому сценарию использования соответствует несколько тестовых сценариев.

59. Модульное тестирование. Junit 4.

Модуль является компонентом системы, который выполняет некую законченную функцию, и который может быть протестирован отдельно от всего вообще. При проектировании системы, мы ее делим сначала на слои, потом на модули. Таким образом, модульное тестирование является наименее масштабным тестированием среди всей системы.

Модуль для тестирования нужно изолировать. Для этого мы вводим заглушки (stub), чтобы исключить какое угодно внешнее воздействие в зависимых модулях. Вместо использования реальных вызывающих модулей, мы вместо этого используем драйверы.

JUnit это фреймворк для проведения модульного тестирования в Java. Он построен преимущественно на аннотациях, и вот его возможности:

- Обозначить метод как тестовый можно с помощью аннотации @Test
- Обозначить метод как исполняемый до/после тестов можно с помощью аннотаций @Before/@After. Можно также задавать порядок исполнения тестов, но подразумевается, что все тесты исполняются в параллель и по-хорошему модульные тесты не должны зависеть от порядка их вызова.
- Различные проверки на соответствие определенным условиям при помощи специальных функций (assertations), формирование тестового покрытия
- Предоставляется журнал тестирования.

60. Интеграционное тестирование. Стратегии интеграции.

Далее по списку в V-образной модели идет интеграционное тестирование. Оно проверяет взаимодействие (!!) модулей/систем между собой. В остальном очень похоже на модульное

тестирование.

В рамках интеграционного тестирования проверяются:

- Вызовы API, сообщения между модулями
- Нажатие кнопок на GUI, доступ к back-end
- Интерфейсы взаимодействия

При составлении плана разработки ПО необходимо учитывать стратегию интеграции, поскольку она прямо коррелирует с календарным планом и сложностью разработки.

Стратегии интеграции:

- Сверху вниз. Сначала мы разрабатываем видимые для конечного пользователя модули, которые выполняют реальные запросы к заглушкам. Это очень распространенная стратегия разработки в бизнес-приложениях, потому что она сразу позволяет предоставить клиенту результаты работы (пользователь часто не видит, что находится под капотом). Из минусов, придется разработать большое число лишнего кода, нужного только для тестирования.
- Снизу вверх. Эта стратегия решает проблему интеграции "сверху вниз", путем того, что сначала разрабатываются модули самого низкого уровня, далее разрабатываются модули более высоких уровней и интеграция таким образом происходит между реальными модулями, однако это усложняет процесс разработки. Данная стратегия нашла свое применение при производстве аппаратуры, там это наиболее актуально.
- Функциональная стратегия (e2e - end to end) — эта стратегия предусматривает наращивание приложений по функциям. Мы сделали функциональность для одного сценария, протестировали его, делаем дальше.
- Backbone (ядро) — здесь мы формируем минимальную рабочую функциональность, далее на ее основе наращиваем наше приложение.
- Big Bang (стратегия большого взрыва). СОБИРАЕМ ВСЕ ЧАСТИ ВОЕДИНО И МОЛИМСЯ! Самая примитивная стратегия интеграции модулей, поскольку она почти никогда не работает. А когда она не работает, вообще непонятно, где возникла ошибка.

61. Функциональное тестирование. Selenium.

Функциональное тестирование — подвид интеграционного тестирования, суть которого заключается в том, что мы полностью разрабатываем некоторую функциональность, которая покрывается тестовым сценарием, а затем проводим тестирование. Особенно популярно при разработке бизнес- и веб-приложений. Оно может быть как ручным, так и автоматизированным, но полностью избежать ручного труда невозможно. Так, например, в случае обработки данных клиента через интерактивную форму добавить еще одно поле, все прошлые тесты сломаются и их придется дорабатывать. Это заметит ручной тестировщик и сформирует соответствующий запрос на изменение.

Для функционального тестирования разработано множество средств автоматизации, которые могут имитировать ввод пользователя с клавиатуры, перемещать указатель и пр. Одно из таких является расширением для браузера Firefox — Selenium. Оно позволяет записать тестовую последовательность

использования интерфейса, затем сохранить это как тестовую программу и использовать ее (даже в других браузерах).

62. Техники статического тестирования. Статический анализ кода.

Проблема динамических тестов проста — нет кода, нечего тестировать (но мы помним про TDD*)

Статические тесты (они же рецензирование) могут быть применены до или во время написания кода, и самое главное преимущество их в том, что они позволяют находить и предупреждать ошибки еще задолго до включения динамических тестов!

Техники статического тестирования заключаются в следующем:

- “Звонок другу”. Мы зовем нашего опытного товарища, который свежим взглядом просмотрит наш код и найдет там какой-то недочет или ошибку. Относится к варианту неформальных ревью. К сожалению, у коллеги тоже есть свои задачи, поэтому он будет доступен в течение очень небольшого числа времени, и этот способ сработает один-два-три(!!!!) раза, но не больше.
- Технический анализ является формальной (IEEE стандарт) техникой проверки артефактов. Он производится в виде собрания под руководством технического лидера, и происходит повторный просмотр с целью выявления плохих практик при разработке.
- Ревью менеджмента является схожей техникой с техническим анализом, только их проводит отдел менеджмента (это другие люди, но примерно та же заноза в заднице)
- Методика сквозного контроля подразумевает превентивные удары по плохому качеству проектирования и разработки. Они заключаются в проведении презентаций, семинаров, где эксперт “ведет” разработчиков по правильным способам разработки, предупреждая магическое мышление разработчиков.
- Методика инспекции берет собой в основу идею о том, что чем более простую задачу дать человеку, тем лучше он с ней справится. Мы берем несколько человек, которые занимаются примитивными вещами (как правило, двумя) и сажаем их рецензировать артефакты, генерируемые командой.

Также стоит упомянуть про автоматизированные средства статического анализа кода. К ним относятся всякие линтеры, они есть для практически любого языка программирования. Они строят грамматическое дерево на основании исходного кода и проверяют частосовершаемые плохие практики использования ЯП (например, предупреждают о NPE, использование fopen без fclose, и пр.)

63. Тестирование системы в целом. Системное тестирование. Тестирование производительности.

После окончания интеграции происходит тестирование системы в целом. Здесь проверяется соответствие заявленным характеристикам. Пререквест: пройденное интеграционное тестирование.

Оно состоит из нескольких частей:

- Системное тестирование
- Альфа-Бета тестирование

- Приемочное тестирование

Системное тестирование производится внутри компании разработчика. Формируется тестовое окружение, формируются сценарии, тестирование происходит от простых сценариев к сложным.

Сначала мы тестируем заявленные возможности ПО в соответствии с утвержденными требованиями. Тестируются только корректные варианты работы, используются только основные сценарии.

Далее мы пытаемся тестировать несколько пользователей в системе или повышенную нагрузку на систему от одного пользователя.

Далее мы тестируем усложненные сценарии. К пользовательскому взаимодействию добавляются заведомо некорректные входные данные, взламываются пользовательские файлы, проверяются умышленные попытки привести программу в состояние сбоя.

Далее мы тестируем, совместима ли наша система с Internet Explorer 6. Если нет, увы и ах, все переделываем. Это называется проверка на совместимость с реальным окружением заказчика. Помимо этого, здесь могут проверяться совместимости с различными версиями third-party библиотек.

И наконец, мы проверяем производительность системы. Смотрим на то, как система справляется в повышенных нагрузках, возможно, при ограниченных размерах памяти и/или процессорного времени.

Тестирование производительности включает в себя все виды тестов, которые образовались в англоязычной литературе в сокращение CARAT:

Capacity — нефункциональные возможности

Они призваны протестировать соответствие нефункциональным требованиям по производительности. Они призваны нагрузить систему до предельно заявленных требований и посмотреть на результат.

Accuracy — точность

Для некоторого ряда систем погрешность допустимых вычислений (например, при использовании численных методов решения задачи, которые были взяты в силу увеличения производительности вычислений) очень важна, и она проверяется здесь.

Response Time — время отклика

Проверяется, насколько система быстро реагирует на поступающие ей запросы. В интерактивных системах это число должно в идеале варьироваться между 1 и 5 секундами, в силу объективных причин человеческого восприятия.

Availability — доступность системы

Выражается в виде коэффициента по формуле:

$Availability = (MTBF - MTBR) / MTBF$, где MTBF - mean time between failure, MTBR - mean time between recovery — среднее время между отказами и среднее время между восстановлением соответственно.

Например, для достижения коэффициента 0,999990487062405 (пять девяток), необходимо, чтобы система в среднем давала сбой каждый год (365 дней = 525600 минут) продолжительностью максимум 5 минут $((525600 - 5) / 525600 = 0.99999)$

Throughput — пропускная способность

Пропускная способность показывает, сколько клиентских запросов может обработать система за единицу времени. Без напряжения современные системы могут вытянуть 100 пользователей в секунду, дальше уже придется заниматься микрооптимизациями.

Существуют системы нагрузочного тестирования, которые позволяют автоматизировать этот этап тестирования. К таким относится, например, Gathling или Apache JMeter.

64. Тестирование системы в целом. Альфа- и бета-тестирование.

После окончания интеграции происходит тестирование системы в целом. Здесь проверяется соответствие заявленным характеристикам. Пререквест: пройденное интеграционное тестирование.

Оно состоит из нескольких частей:

- Системное тестирование
- Альфа-Бета тестирование
- Приемочное тестирование

Альфа и Бета тестирование производится уже непосредственно пользователями под контролем разработчика.

При альфа тестировании мы приглашаем пользователя к нам в компанию и даем ему пользоваться нашей системой. Все происходит за нашим подготовленным окружением, под нашим контролем, "пользователи пьют наше кофе, заразы...". Пользователи могут обнаружить дополнительные косяки, которые не предусмотрели разработчики (уволить их после этого желательно)

При бета тестировании мы устанавливаем систему уже на конечном компьютере пользователя, но тестирование все еще проводится под нашим контролем.

Как только мы собрали фидбек от пользователей, внесли возможные исправления, наступит приемочное тестирование.

65. Аспекты быстродействия системы. Влияние средств измерения на результаты.

Системный и архитектурный аспект

- Присутствует влияние архитектуры (монолитная, n-tier)
- Свое влияние вносит кластеризация, виртуализация

Низкоуровневый аспект

- Характеристика аппаратуры (частота, количество ядер, скорости кешей, время доступа к дисковой подсистеме)

Программный аспект

- Выбор алгоритма
- Многопоточность
- Человеческий фактор

Средства измерения могут быть

- Неинтрузивные
- Слабоинтрузивные
- Сильноинтрузивные

Разница в том, что неинтрузивные средства измерения не вносят никакого влияния на результаты работы. В физическом мире, если вставить градусник в воду, вода будет от него нагреваться, как только установится состояние теплового равновесия, тогда мы сможем получить температуру, но это средство измерения внесло серьезные изменения в результат. Поэтому почти все измерительные приборы в физике являются сильноинтрузивными.

В программной инженерии есть пример неинтрузивных средств измерения — счетчики операционной системы. Они все равно будут собираться, хотите вы того или нет, эти издержки были в вашей операционной системе всегда.

Прим. ред: в этом билете можно расписать про то, что анализ производительности это научный эксперимент и бла-бла-бла, так как это есть в сопутствующем слайде, но я не считаю, что это является частью вопроса.

66. Ключевые характеристики производительности.

- Время отклика (время от момента запроса к системе до получения первых ответов от нее)
- Время полного обслуживания (время от момента запроса к системе до получения последнего ответа от нее)
- Пропускная способность (максимальное количество одновременно работающих пользователей в системе за единицу времени)
- Утилизация (%util, %busy) и ожидание (%wait %idle) ресурса
- Точка насыщения производительности (момент предельной нагрузки на систему, после которой производительность будет только падать с ростом нагрузки)
- Масштабируемость (показатель возможностей горизонтального (количественного) расширения системы)
- Эффективность, КПД (отношение полезной работы к общей)
- Java (прим. ред.: фу): Скорость запуска и memory footprints (стратегии работы с памятью)

67. Нисходящий метод поиска узких мест.

- Исключить ошибки аппаратуры и системного администратора (Ой, а у нас тут диск перегорел уже 10 лет как) (Ой, а у нас администратор криворукий, настроил 16 Java-машин, каждая резервирует себе по 4GB памяти...)
- Исключить ошибки операционной системы. Мониторинг CPU, I/O подсистемы, сети, каналов передачи, интерфейсов
- Исключить ошибки приложения. Пофиксить алгоритмические проблемы, проблемы API, исключить взаимные блокировки и пр.
- Исключить ошибки микроархитектуры. Пофиксить кеш-промахи, избавиться от пузырьков конвейера и пр.

68. Пирамида памяти и ее влияние на производительность.

Дубликат билета №28 (Характеристики запоминающих устройств. Пирамида памяти) из экзамена по Основам Профессиональной Деятельности.

69. Мониторинг производительности: процессы.

Процесс может исполняться на CPU (On CPU), в пользовательском (User) или системном (System) режиме (он же режим ядра), быть готовым к исполнению (Runnable) или ждать чего-то/находиться в блокировке (Wait). Допустим, у нас есть 2 процесса. Пусть в некоторый момент начала наблюдения исполняется процесс 1. На CPU исполняется пользовательская программа. Раз в некоторое время (см. процесс квантирования в курсе Операционных Систем) генерируется прерывание от часов и начинает исполняться системный код. Во время работы системного кода наращиваются счетчики производительности, производится подготовка к следующему кванту времени. Далее управление возвращается пользовательской программе.

Как только процессу понадобилось чего-то подождать (например, работу устройства ввода-вывода), сразу после этого, во избежание простаивания ресурсов процессора происходит процесс диспетчеризации. На место работы CPU встает второй процесс, которому отдадут поток управления, и который будет работать до следующего прерывания от часов (как и в случае с первым процессом). Как только от контроллера ВУ поступит прерывание о готовности данных, процесс 1 вернется в состояние Runnable. Как только наступит следующее прерывание от часов, системе придется выбрать, какой из процессов стоит дальше пустить на CPU. В современных ОС это делается при помощи честной системы приоритизации. Если процесс отработал весь квант времени, его приоритет понижается, приоритет остальных процессов, наоборот, повышается за каждый квант времени, за который процесс простаивает в состоянии Runnable.

В случае на картинке может быть 3 сценария:

- Процесс 1 оказался более приоритетным. Произойдет involuntary context switch (вытесняющая многозадачность), мы отбираем у процесса его время и отдаем ее другому процессу.
- Процесс 2 оказался более приоритетным и он доработал до блокировки. В таком случае процесс передачи управления при помощи dispatch от процесса 1 к процессу 2, описанного ранее, повторится с точностью до обратного.

- Процесс 2 оказался более приоритетным и доработал полностью. В таком случае произойдет voluntary context switch.

70. Мониторинг производительности: виртуальная память.

Виртуальная память это механизм управления памятью компьютера, при котором программа работает с виртуальным адресным пространством, которое привязывается к физическим адресам, давая исполняемой программе иллюзию того, что она единственная (и неповторимая) исполняется на системе, но более того, виртуальная память позволяет разделять общие участки кода (например, `libc` или ядро операционной системы) между разными процессами, позволяя экономить физическую память. К тому же, механизм виртуальной памяти позволяет перемещать неиспользуемые страницы памяти на более медленный накопитель, экономя место в оперативной памяти (своппинг).

Современная операционная система старается держать некоторое количество страниц памяти всегда свободными (для каких-то сверх экстренных нужд, которые могут в ней возникнуть). Когда количество свободных страниц уменьшилось до определенного порога, запускается процесс, который начинает сканирование виртуальной памяти на предмет неиспользуемых страниц для записи (с помощью проверки `dirty bit` у страницы) и для чтения (с помощью проверки обращений к странице). Неиспользуемые страницы сбрасываются в своп или даже удаляются из памяти вовсе (если их можно загрузить обратно с жесткого диска).

Если свободной памяти все равно становится меньше, то система увеличивает т. н. `scan rate`, т. е. количество проверок виртуальной памяти становится все больше и больше, система пытается сбросить как можно больше страниц памяти.

Если это все равно не помогает, операционная система начинает подвешивать приходящие ей запросы на аллокацию памяти, также при данном состоянии вы можете услышать звуки бурной деятельности вашего жесткого диска, так как операционная система постоянно работает со страницами в памяти и увеличивается количество `page fault`'ов.

Если это тоже не помогает, и количество памяти уменьшается, система начнет отвергать запросы на аллокацию памяти. Это означает, что `malloc` в C будет возвращать `NULL`, а в JVM (прим. ред.: фу) будет сгенерирована ошибка `OutOfMemoryError`.

В Linux мониторинг виртуальной памяти производится при помощи команды `sag -B`. Эта команда даст вам знать о количестве загружаемых/выгружаемых/освобождаемых страниц в файл подкачки в секунду, количество промахов в страницах, `scan rate` и другую полезную информацию о виртуальной памяти.

71. Мониторинг производительности: буферизированный файловый ввод-вывод. (??)

Пусть в нашей программе появилось обращение к файлу, чтобы что-то оттуда прочитать. Рано или поздно это дойдет до системного вызова к подсистеме VFS в ядре (VFS - Virtual File System). Она обратится к `Directory Name Lookup Cache (DLNC)` для преобразования имени файла в его номер `inode` в необходимой файловой системе. Далее операция чтения будет работать с кешем. Этот кеш хранит в себе подгруженные с диска данные, любые изменения раз в какое-то время будут сгружаться обратно на диск. Если был обнаружен промах в кеше, загружается специальный файл драйвера работы с

конкретной файловой системой и далее уже ведется работа по чтению данных с конкретной аппаратуры.

Все файловые системы имеют абстрактный уровень — VFS

Для каждой файловой системы существует ряд параметров:

- Количество чтений и записей в секунду
- Объем прочитанных и записанных килобайт в секунду
- Среднее время запросов
- Время ожидания
- Время обслуживания
- Процент занятости (утилизации) устройства

Получить эту статистику в Linux можно с помощью команды `iostat -x`

72. Мониторинг производительности: Windows и Linux.

Мониторинг производительности в Windows представляет собой встроенное средство просмотра всей основной информации об использовании компьютера — Windows Task Manager.

Более подробная статистика с продвинутыми счетчиками есть в коммерческом и свободно распространяемом ПО, наиболее продвинутым средством является разработка Microsoft SysInternals.

В Linux ситуация выглядит веселее:

Для того, чтобы получить информацию из счетчиков Linux, вам для этого помогут команды в терминале, например, `vmstat`, `top`, `perf`, `sar` и другие:

- `vmstat` соберет основную информацию из всех основных подсистем Linux (CPU, Swap, IO, Memory), раз в определенное время в зависимости от аргументов командной строки
- `top` позволяет следить за процессами в системе, например, за их приоритетом, занимаемой CPU и памятью (виртуальная, физическая, разделяемая с другими процессами) и другую информацию. В статусной строке можно увидеть большое число общесистемных характеристик
- Утилита `sar` является одной из первых утилит в ОС Unix (по состоянию на 2024, в macOS Ventura этой команды не найдено даже в POSIX-совместимом shell) по мониторингу производительности, позволяет мониторить основную системную информацию ядра. Некоторые метрики собираются устаревшим способом, поэтому команду стоит использовать с осторожностью
- Отдельно стоит выделить утилиту `perf`, которая позволяет собирать огромное число метрик не только ядра, но и приложения, запущенного под этой утилитой. `Perf` умеет работать с большим числом счетчиков производительности процессора для сбора и поимки таких событий, как, например, промахи кеша.
- `Star` позволяет установить точки в ядре, чтобы агрегировать информацию о работе подсистем ядра.

73. Системный анализ Linux "за 60 секунд". (достался мне на экзамене)

Практический подход "Системный анализ Linux за 60 секунд" представляет собой последовательность команд, исполнение которых быстро введет человека в курс дела о том, что происходит в системе:

- `uptime` покажет `load average` — количество готовых к выполнению процессов в очереди на диспетчеризацию за 1, 5 и 15 минут. Если процессов много, система не справится с нагрузкой
- `sudo dmsg | tail` — часто в системе есть какие-то явные сбои, взглянув на которые можно понять, куда двигаться дальше. Эта команда позволяет получить логи системных сбоев.
- `vmstat 1` — следом мы получаем представление о том, есть ли у нас свободная память, происходит ли процесс пэйджинга/своппинга, как утилизируется центральный процессор
- `mpstat -P ALL 1` — смотрим на распределение процессов по центральному процессору, так как возможная ситуация, когда занят только один процессор (или ядро), остальные простаивают.
- `pidstat 1` — проверяем наиболее "горячие" процессы, смотрим статистику
- `iostat -x 1` — проверяем работу подсистемы ввода-вывода
- `free -m` — проверка исчерпания кешей или буферов
- `sar -n DEV 1` — смотрим сетевую статистику по интерфейсам
- `sar -n TCP, ETCP 1` — смотрим сетевую статистику по соединениям
- `top` — в завершение, эта команда является швейцарским ножом (шутка, швейцарским ножом является утилита `perf`) для системного администратора, многие названные выше характеристики можно посмотреть в едином месте с помощью как раз данной утилиты

74. Создание тестовой нагрузки и нагрузчики.

Средства мониторинга — сильноинтрузивные

В корпоративных системах бывает строго настрого запрещено прикасаться к боевой системе (она работает и уйди от нее подальше, и не приближайся):

Они могут создать дефекты в работе системы → BSOD/Kernel Panic (боевая система упала по вине этих проклятых тестировщиков, из-за них мы теряем деньги)

Решение: создать такую же систему как основную и нагрузить ее "так же, как и основную":

- Для этого есть средства создания синтетической нагрузки (Apache JMeter, Gathling)
- Средства, которые записывают реальную нагрузку (присутствуют на боевой системе), чтобы помочь ее воспроизвести

Все-таки стоит отметить, что синтетическая нагрузка всегда будет отличаться от реальной, насколько близко (с помощью большого числа гибко настраиваемых параметров) мы бы не пытались это настроить.

75. Профилирование приложений. Основные подходы.

После того, как мы выяснили, что вся наша аппаратура в порядке, а операционная система и все ее подсистемы настроены хорошо, но производительность нас по-прежнему не устраивает, в дело вступает профилирование приложения. Если у нас есть исходные коды, мы можем в них провалиться в поиске алгоритмических дефектов.

С помощью профилировщика можно узнать:

- Время исполнения метода
- Объем созданных объектов в памяти
- Состояние многопоточности, захваченные блокировки и пр.

Существуют два основных подхода к профилировке:

- Диагностические точки можно внедрять в сами функции из указанного набора (в начале и в конце функции). Это интрузивный подход, мы меняем исходный код и заставляем тратить время на дополнительное исполнение. К тому же мы тратим время на поиск этих самых точек для внедрения профилирования
- Используем прерывания программы, и во время прерывания собираем интересующую нас информацию, например, анализируем стек вызовов, смотрим на состояние кучи.

76. Компромиссы (trade-offs) в производительности.

Любая борьба за производительность сопряжена с т. н. trade-off'ами (компромиссами):

- Различные подсистемы ЭВМ взаимосвязаны: чем быстрее мы имеем доступ к данным, тем больше памяти они занимают (linear search vs indexing)
- Выбор алгоритма и архитектуры может существенно ускорить приложение, либо сделать его максимально компактным (см. Алгоритмы и структуры данных)

Примеры:

- Добавление кеша не обязательно делает доступ к памяти быстрее (нам требуется время на перегон памяти между промежуточным звеном между процессором и памятью)
- Бесконечное увеличение кеша бесполезно — настанет момент времени, когда добавление кеша не будет добавлять прирост к производительности, при этом будет расти стоимость такого решения.
- Выбор алгоритма сортировки на маленьких и больших объемах данных может существенно отличаться. Иногда нам будет рациональнее взять асимптотически более тяжелый алгоритм в угоду его простоты или компактности.

77. Рецепты повышения производительности при высоком %SYS.

Высокая нагрузка на ввод вывод (сеть, диск,...)

- Рассмотреть возможность меньше обращаться к подсистеме ввода-вывода. Сжимать данные, оптимизировать систему.
- Купить буферы, более быстрые устройства ввода-вывода
- Синхронизировать размер блока передачи между ОС и устройством ввода-вывода

Высокая степень работы планировщика (постоянные контекст свичи, высокий uptime)

- Уменьшить количество потоков, наладить работу ОС
- Забиндить потоки к своим ядрам

Paging/Swapping

- Выдать больше памяти системе, ограничить память процессам
- Запретить swapping

Ядро пишут тоже люди

- Попытаться разобраться в конфигурации ядра
- Сообщить о какой-то проблеме при ее обнаружении в багтрекер

78. Рецепты повышения производительности при высоком %IO wait.

Проблемы приложений

- Рассмотреть возможность меньше обращаться к подсистеме ввода-вывода. Сжимать данные, оптимизировать систему.
- Синхронизировать размер блока передачи между ОС и устройством ввода-вывода

В системе выделено мало кеша

- Расширить кеш
- Настроить его грамотное использование, исключить лишние промахи

Проблема аппаратуры

- Купить более быстрые устройства ввода-вывода

79. Рецепты повышения производительности при высоком %Idle.

Программа убивается в один поток на одном ядре (или около того)

- Распараллелить все, что можно
- Расширить пулы потоков

Анализ блокировок, исключить потенциальные взаимные блокировки

- По возможности использовать lock-free алгоритмы

Проблемы внутри ОС, ядро тоже пишут люди

- ОС имеет слишком много блокировок — мониторинг средствами ОС
- Настройка параметров в ядре
- Поиск дефектов ядра в багтрекере, сообщить о проблеме, если таковая была обнаружена

80. Рецепты повышения производительности при высоком %User.

Проблемы приложений: не всегда это означает полезную работу программы

- Использовать алгоритмы с меньшей алгоритмической сложностью
- Не создавать по новой лишние объекты в памяти, аллоцирование памяти это невероятно дорогая операция, повторное использование объектов
- Избавиться от активных опросов и ожиданий в цикле (busy-waiting)
- Искать промахи в кешах (заменять структуры данных) или в трансляторе виртуальной памяти (использовать больший размер страничек памяти)
- Для исключения остывания кешей — биндить потоки и процессы к конкретным ядрам/процессорам
- Оптимизации на уровне компилятора или языков более низкого уровня
- Повысить аппаратные характеристики, увеличить тактовую частоту, количество ядер, использовать GPU (прим. ред.: NVIDIA CUDA!) или иную вычислительную технику

Спасибо за прочтение! Удачной подготовки!

14/06/2024 23:36:51 by Zerumi